

TECHNICAL PRESENTATION

攻撃フレーム・当たり判定をエディタ上で調整できる 2D格闘ゲーム

1フレーム単位の攻撃制御と、ScriptableObject×独自拡張エディタによる
調整しやすい攻撃データ管理の実装

横浜デジタルアート専門学校 ゲーム科3年 / 田上 ヒデオ

01. 作品概要と仕様

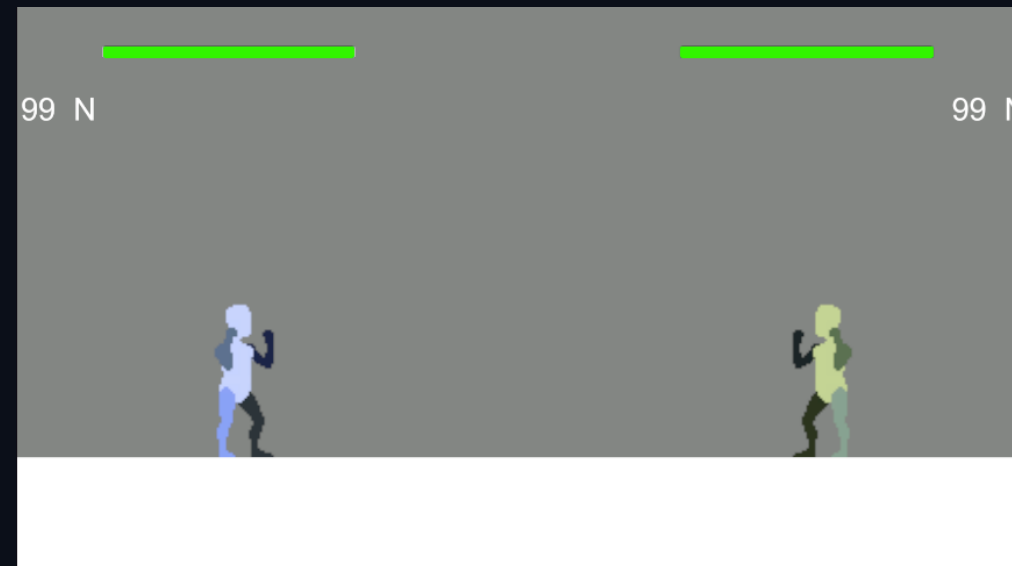
Core Concept & System Specs

ゲーム概要と基本仕様

攻撃の発生・持続・硬直による駆け引きを重視した2D格闘ゲーム

プレイヤーの入力、攻撃状態、当たり判定、ガード、ノックバックを制御し、対戦ゲームらしい操作感を目指しました。

項目	内容
ジャンル	2D格闘ゲーム
開発期間	5ヶ月
制作人数	1人
使用技術	Unity / C#
主な実装	通常攻撃、ガード、ノックバック、当たり判定、入力ログ表示
工夫点	攻撃データ管理、フレーム制御、独自EditorWindow



02. システムアーキテクチャ

Game System & Architecture Design

コアゲームマネジメント設計

⚙️ GameManager.cs

対戦前、バトル中、KOなど、ゲーム全体の状態を管理するクラスです。外部から不用意に状態が変更されないようにし、ゲーム進行をまとめて管理できるようにしました。

🔄 State Pattern



🧩 CharaBase.cs

Player.csとEnemy.csに共通する移動、被弾、ステート変化などの処理をまとめた基底クラスです。共通処理をまとめることで、キャラクターごとの処理を追加しやすい構成にしました。

→ Player.cs (キー入力制御)

→ Enemy.cs (AI行動ロジック)



🔌 PlayerHpSlider.cs

HP表示を担当するUI用クラスです。ゲームロジックとUI処理を分けることで、HP変更時の表示更新を管理しやすくしました。

🔌 Event-Driven UI

対戦中における状態遷移とゲームフロー

2. IN-BATTLE

対戦中の状態です。入力、移動、攻撃、ガード、HitBox、HurtBoxの更新をフレーム単位で処理します。

4. RESULT & RESET

KO後に勝敗判定し、ラウンドのリセット処理を行います。

1. STANDBY

対戦開始前の初期化状態です。
ScriptableObjectからキャラクターごとの攻撃データやスプライト情報を読み込みます。

3. HIT & CONFLICT

HitBoxとHurtBoxが重なった際に、ヒットストップ、ダメージ、ノックバックを適用します。

03. コアロジックとフレーム制御

Core Logic & Frame-Perfect Sync

手触りを作るためのフレーム制御ロジック

対戦ゲームらしい手触りを作るためのフレーム制御

攻撃ごとに発生・持続・硬直を設定し、1フレームごとにSprite切替・HitBox更新・Hit判定を行う構成にしました。

攻撃タイミングと当たり判定を細かく調整できるため、プレイヤーが納得しやすい接触判定を目指しました。

 totalFrameを基準にしたフレーム進行: 現在のフレームを基準に、攻撃の発生・持続・硬直を制御。

 着地自動キャンセル: 空中技が着地した瞬間に、判定を持続せず即座に接地後隙に強制移行。

```
Attack.cs

// 攻撃データの全体フレームを完全同期ループ制御
while (attackCnt < attack.totalFrame) {
    if (hitFlg) break;

    // 空中攻撃時の着地自動キャンセル処理
    if (jumpAttackFlg && groundFlg) {
        DisableAllBoxes();
        HurtBox[0].enabled = true;
        animator.enabled = true;
        break;
    }
    sp.sprite = attack.spriteAll[attackCnt]; // スプライト変更
    SetAttack(attack);                       // 当たり判定更新
    attackCnt++;
    yield return new WaitForSeconds(GameManager.frame);
}
```

04. 内製開発ツール 「HitBox Editor」

Custom Editor Tool Design

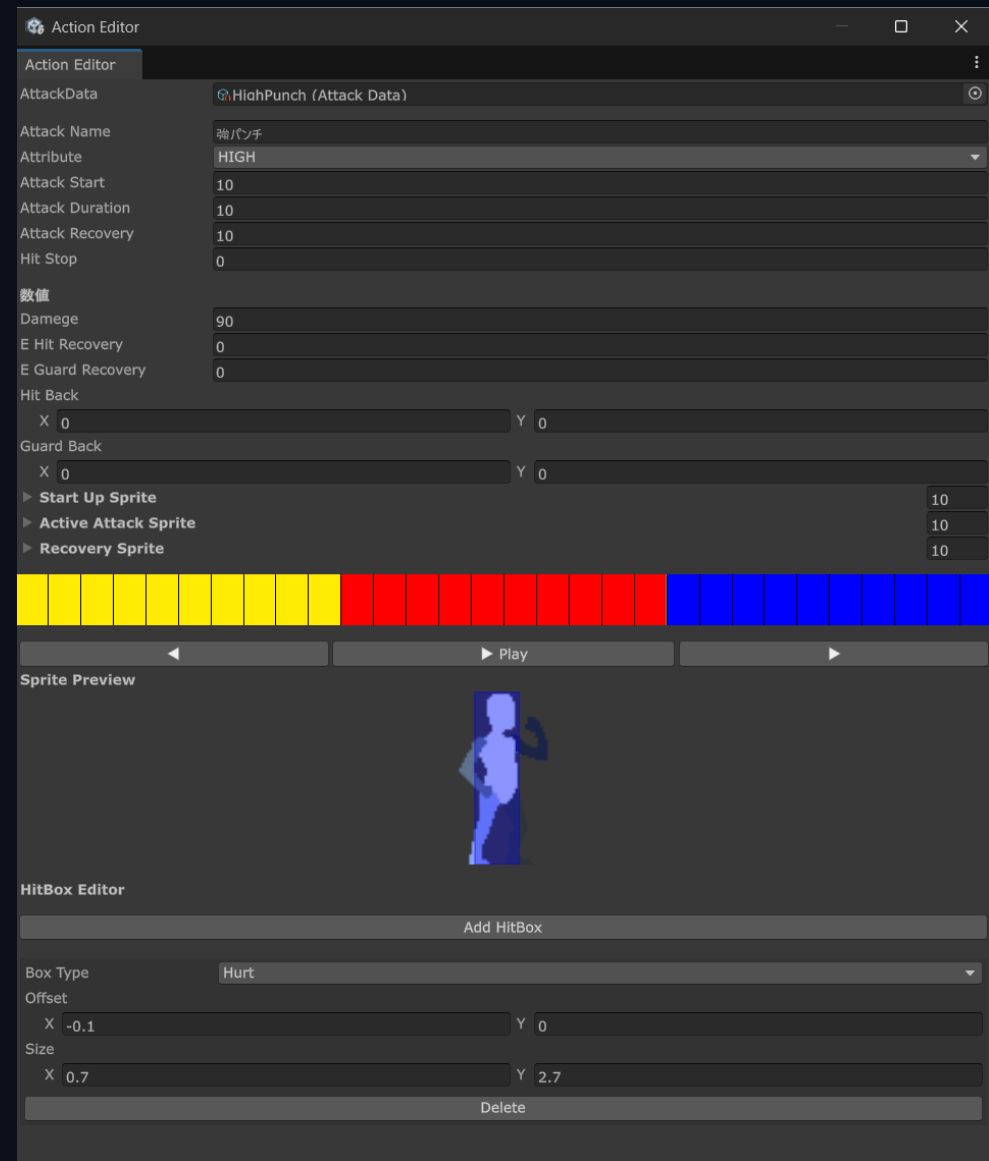
攻撃調整を効率化する独自エディタを作成

専用エディタ上で確認・編集できる快適な開発環境

攻撃判定の調整では、フレームごとの判定位置やサイズを細かく確認する必要があります。

そこで、UnityEditorを使って専用の編集ツールを作成しました。

スプライトを確認しながら、HitBoxの位置・サイズ・発生タイミングを編集できるようにしています。



05. 課題解決と開発効率化の成果

Technical Challenges & Pipeline Results

開発効率化を可能にするデータ駆動ワークフロー

⚠ 従来の手入力調整（課題）

- コード値やInspectorの直接手入力：
当たり判定のピクセル座標やタイミング数値を感覚で打ち込む必要があり、パラメータ調整が困難。
- 再生テストの頻繁な繰り返し：
コライダー変更がゲームビューにどう影響するか確認するために、毎回ゲーム再生を起動する時間的ロス。
- コンパイル待ちと再生停止：
ズレや不具合が発覚するたびにゲーム再生を停止し、再定義して再読み込みする非効率なイテレーション。

✔ 本エディタによる効率化フロー

- タイムライン上での視覚的編集：
色分けされた専用GUIバーをドラッグ&ドロップでスライドさせるだけで、直感的に1フレームを調整可能。
- エディター上でのプレビュー：
ゲームを再生しなくても、エディットモードのままスプライト上に当たり判定の位置やサイズを視覚的に確認。
- ScriptableObjectとの完全連携：
設定結果は即座にAttackDataにアセット保存され、C#コードの書き換えや再コンパイルが一切発生しない設計。

攻撃データ管理・調整における具体的な成果



コード変更なしでのデータ調整

ダメージ値、ノックバック量、ガード硬直などを
ScriptableObject (AttackData.asset) に分離し、
コード変更せずに攻撃データを調整できるようにしました。



フレーム判定の視覚的確認

Sprite表示とHitBox / HurtBoxをエディター上で
確認できるようにし、見た目と接触判定のズレを
調整しやすくしました。



攻撃追加・調整が容易な設計

技ごとのAttackDataを作成・複製することで、
既存のロジックを大きく変更せずに
攻撃を追加できる構成にしました。

この制作でアピールできること



フレーム制御を理解して実装

攻撃の発生・持続・硬直をフレーム単位で管理しました。Sprite切替とHitBox 更新も同期させ、納得感のある接触判定を目指しました。



データ駆動による攻撃パラメータ管理

ScriptableObjectを活用し、攻撃パラメーターをコードから分離。コード修正なしで調整できる構成にしました。



Editor拡張による開発効率化

HitBox / HurtBoxをエディター上で視覚的に確認・編集できるツールを作成し、調整作業の負担を軽減しました。



拡張を活用したクラス設計

PlayerとEnemyの共通処理をCharaBaseに集約。オブジェクト指向を意識し、仕様変更や機能追加に対応しやすい設計にしました。